Laporan Tugas Pratikum

Jobsheet 7

Pemrograman Framework

Oleh:

Satrio Ahmad Ramadhani

2341720163

TI-3B/22

D-IV TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

JOBSHEET PRAKTIKUM

Mata Kuliah : Pemrograman Framework

Program Studi : D4 (Sarjana Terapan)

Topik Praktikum : API Routes pada Next.js dan Integrasi Firebase (Fullstack Next.js)

Alokasi Waktu : 4 × 50 menit

A. Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa mampu:

1. Memahami konsep API Routes pada Next.js.
2. Membuat API sederhana menggunakan Next.js.
3. Mengirim response JSON dengan status code.
4. Mengambil data API di sisi frontend menggunakan fetch.
5. Mengintegrasikan Firebase Firestore sebagai database.
6. Mengelola environment variable (.env.local).
7. Menampilkan data dinamis dari database ke halaman web.

B. Dasar Teori Singkat

1. API Routes di Next.js

Next.js tidak hanya framework frontend, tetapi juga dapat berfungsi sebagai backend melalui API Routes.

File API diletakkan di:

- pages/api/

Contoh:

- pages/api/hello.js

Endpoint dapat diakses melalui:

- /api/hello

---

2. Firebase Firestore

Firebase adalah layanan backend dari Google yang menyediakan:

- Database (Firestore / Realtime DB)
- Authentication

- Storage (File)

Firestore bersifat NoSQL dan cocok untuk aplikasi modern.

C. Alat dan Bahan

Perangkat Lunak

- Node.js (LTS)
- NPM
- Visual Studio Code
- Browser
- Akun Google (untuk Firebase)

Prasyarat

- Project Next.js (Pages Router) sudah tersedia
- Aplikasi dapat dijalankan (npm run dev)

D. Langkah Kerja Praktikum

Langkah 1 – Menjalankan Project

- npm run dev

Akses:

- http://localhost:3000

Langkah 2 – Membuat API Produk

1. Buat file pad pages/api/produk.js

2. Tambahkan data statis:

6.praktikum-ApiRoutes > my-app > src > pages > api > TS produk.ts > ...

```typescript
// Next.js API route support: https://nextjs.org/docs/api-routes/introduction
import type { NextApiRequest, NextApiResponse } from "next";

Windsurf: Refactor | Explain
type Data = {
    status: boolean;
    status_code: number;
    data:{
        id: string;
        nama: string;
        harga: number;
        ukuran: string;
        warna: string;
    }[];
};

Windsurf: Refactor | Explain | Generate JSDoc | X
export default function handler(
    req: NextApiRequest,
    res: NextApiResponse<Data>,
)
    {
        const data =
        [
            {
            id: "1",
            nama: "Kaos Polos",
            harga: 10000,
            ukuran: "L",
            warna: "merah"
            },
            {
            id: "2",
            nama: "Kaos Berlengan Panjang",
            harga: 15000,
            ukuran: "M",
            warna: "biru"
            },  ];
        res.status(200).json({status:true,status_code:200,data:data});
    };
```

3. Akses: http://localhost:3000/api/produk

VPN localhost:3000/api/produk


```
{
    "status": true,
    "status_code": 200,
    "data": [
        {
            "id": "1",
            "nama": "Kaos Polos",
            "harga": 10000,
            "ukuran": "L",
            "warna": "merah"
        },
        {
            "id": "2",
            "nama": "Kaos Berlengan Panjang",
            "harga": 15000,
            "ukuran": "M",
            "warna": "biru"
        }
    ]
}
```


4.

Langkah 3 – Fetch Data API di Frontend

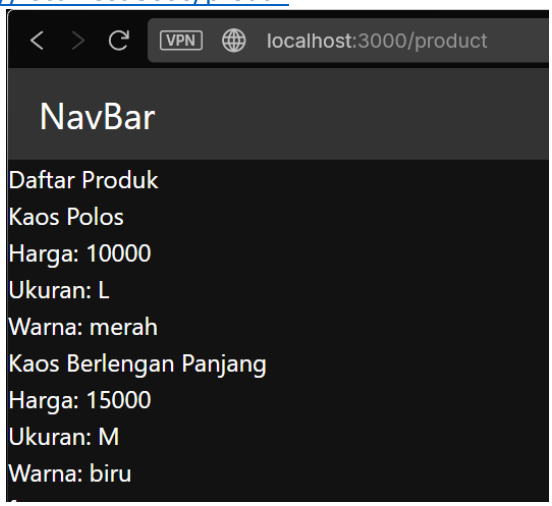
1. Buka pages/product/index.tsx

   - Modifikasi kode
     - Tambahkan useEffect() dan comment useEffect untuk isLogin

6.praktikum-ApiRoutes > my-app > src > pages > produk > index.tsx > kategori

```
import { useRouter } from "next/router";
import { useEffect, useState } from "react";

Windsurf: Refactor | Explain
type ProductType =
{
    id: string;
    nama: string;
    harga: number;
    ukuran: string;
    warna: string;
  }
Windsurf: Refactor | Explain | Generate JSDoc | X
const kategori = () => {
  // const [isLogin, setIsLogin] = useState(false);
  // const { push } = useRouter();
  const [products, setProducts] = useState([]);

  // useEffect(() => {
  //   if (!isLogin) {
  //     push("/auth/login");
  //   }
  //   },[]);

  useEffect(() => {
    fetch("/api/produk")
      .then((response) => response.json())
      .then((responsedata) => {
        //console.log("Data produk:", responsedata.data);
        setProducts(responsedata.data);
      })
      .catch((error) => {
        console.error("Error fetching produk:", error);
      });
  }, []);


  return (
  <div>
    <h1>Daftar Produk</h1>
    {products.map((product:ProductType) => (
    <div key={product.id}>
      <h2>{product.nama}</h2>
      <p>Harga: {product.harga}</p>
      <p>Ukuran: {product.ukuran}</p>
      <p>Warna: {product.warna}</p>
    </div>
  ))}
  </div>
  );
};

export default kategori;
```

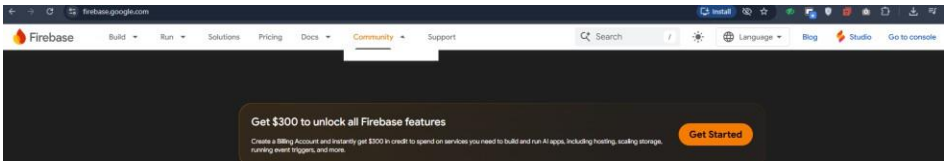
2. Jalankan browser http://localhost:3000/produk



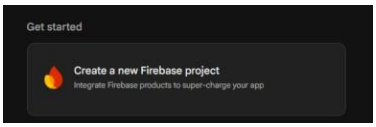
E. Integrasi Firebase

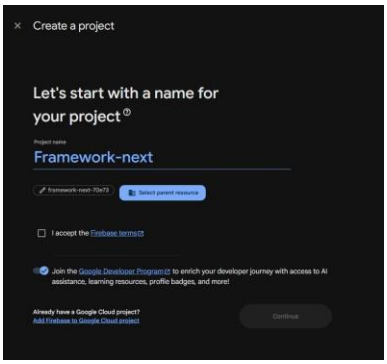
Langkah 5 – Setup Firebase

1. Buka Firebase Go To Console (login dengan login google)

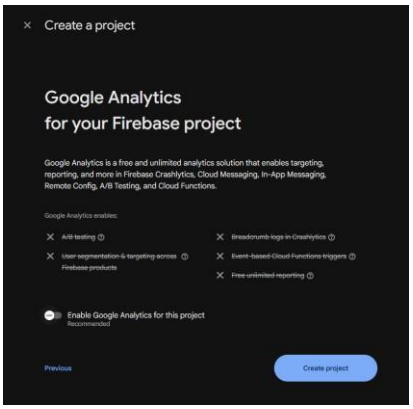


2. Buat project baru

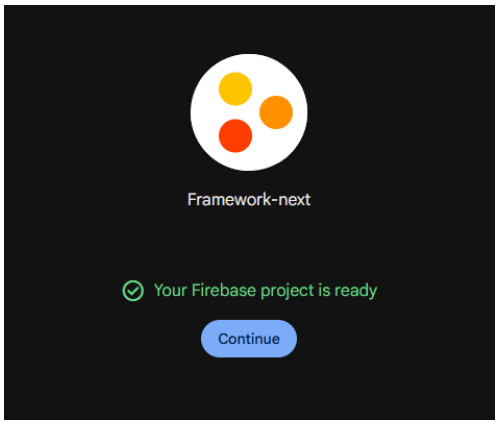


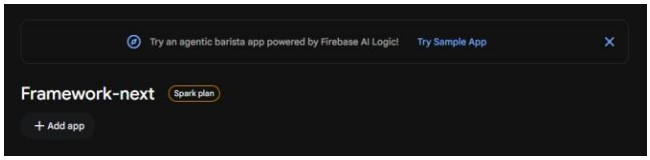


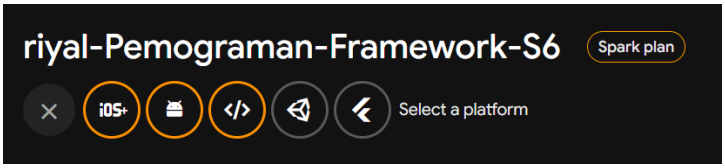
Note : jangan lupa select parent resource nya



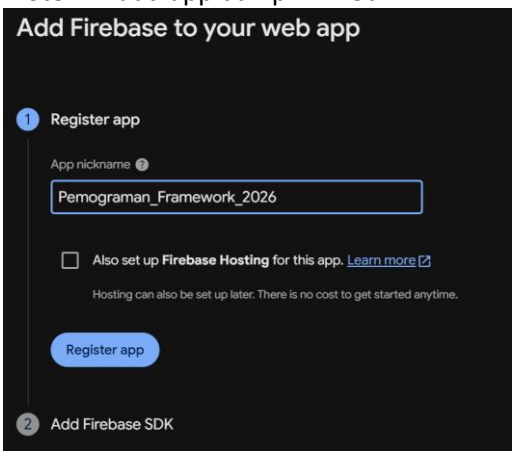
Note : klik create project dan disable google analytics



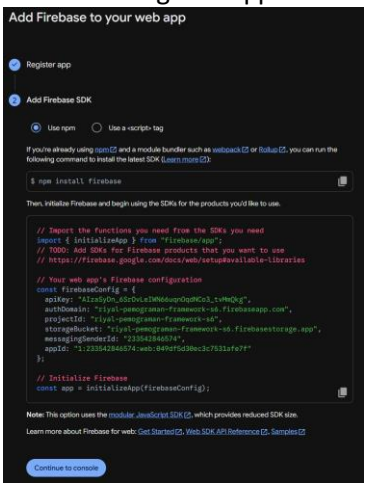




Note klik add app dan pilih web

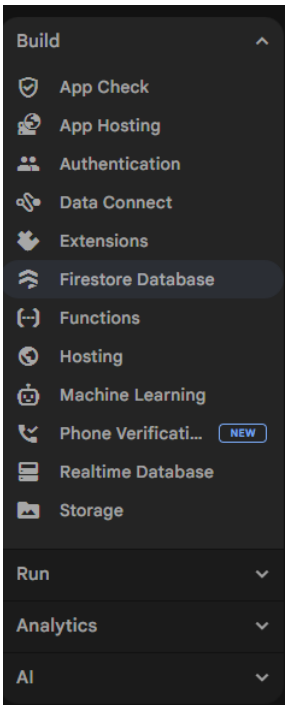


Note : klik register app

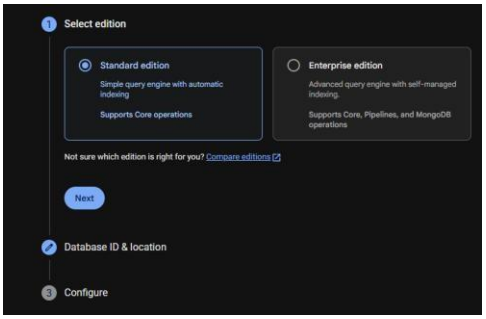


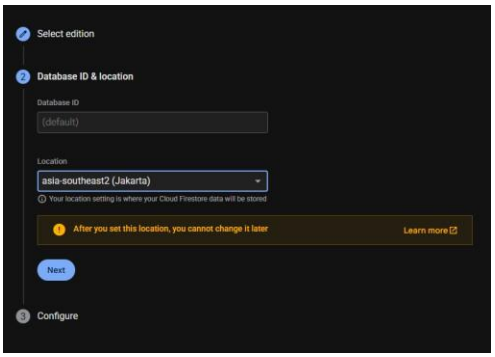
Note : klik continue to console

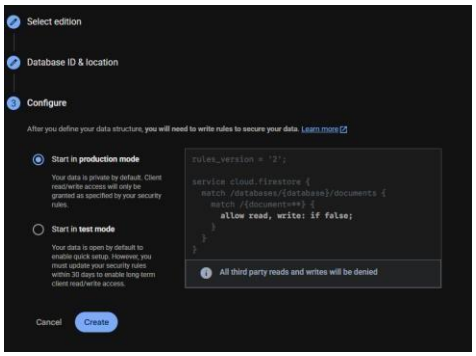
3. Aktifkan Firestore Database

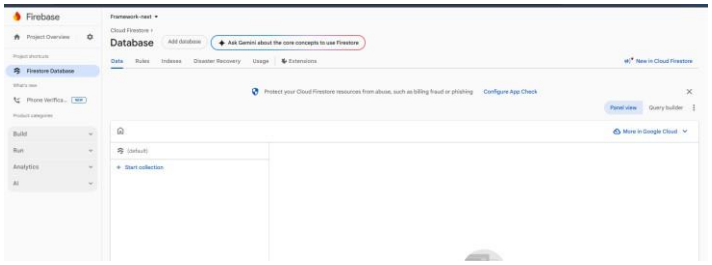


Note : klik create database

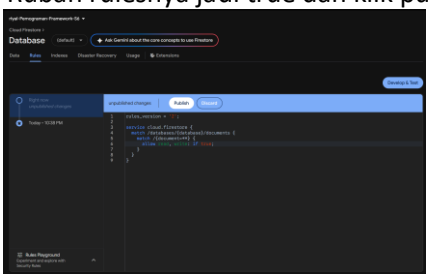




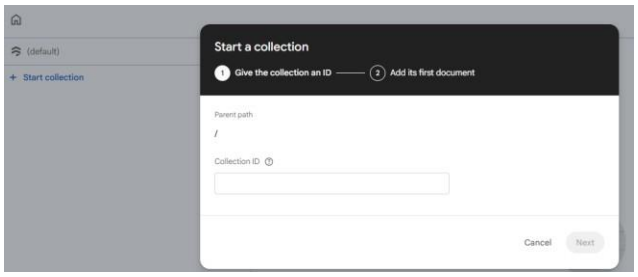




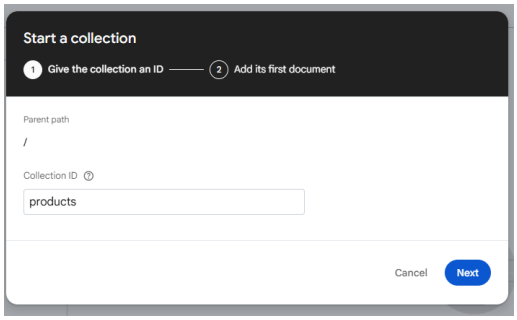
Rubah rulesnya jadi true dan klik publish



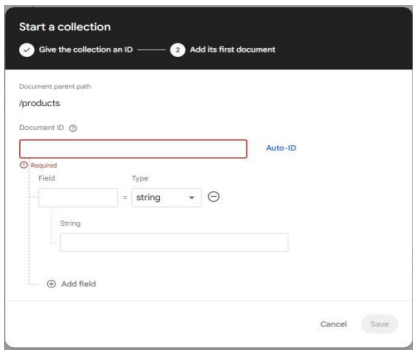
4. Buat collection:



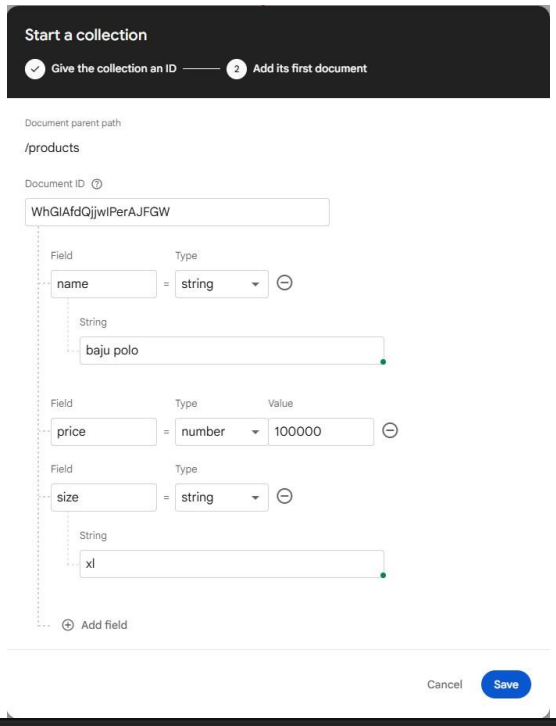
5. products

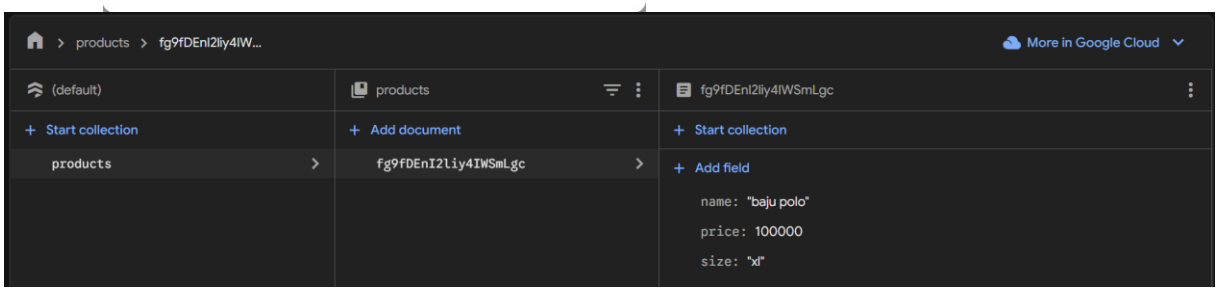


Gunakan auto-id



6. Tambahkan field:





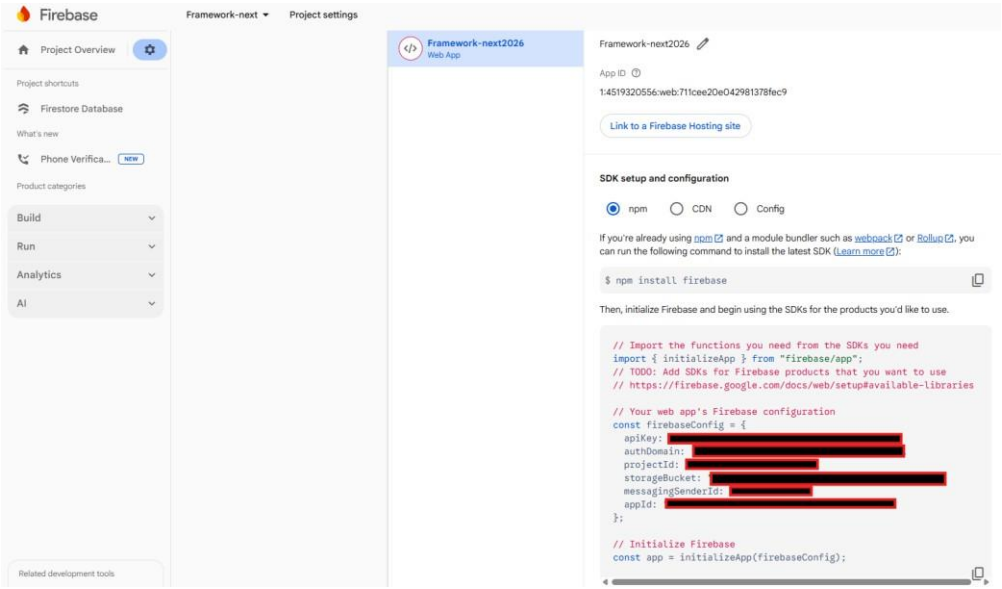
Langkah 6 – Install Firebase

1. npm install firebase

```
PS D:\1.MateriKuliah\2026-ganjil\PemogramanFrameWork\Code\6.praktikum-ApiRoutes\my-app> npm install firebase

added 79 packages, and audited 426 packages in 33s

142 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

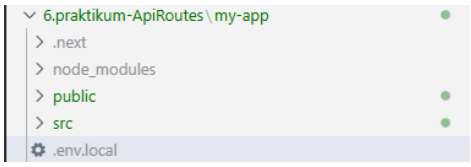
2. Buat folder dan file ts pada pages utlis/db/firebase.ts

3. Copy paste yang ada pada kotak merah ke file firebase.ts



Langkah 7 – Konfigurasi Environment Variable agar credensial firebase tidak dapat dilihat saat dipush di repository

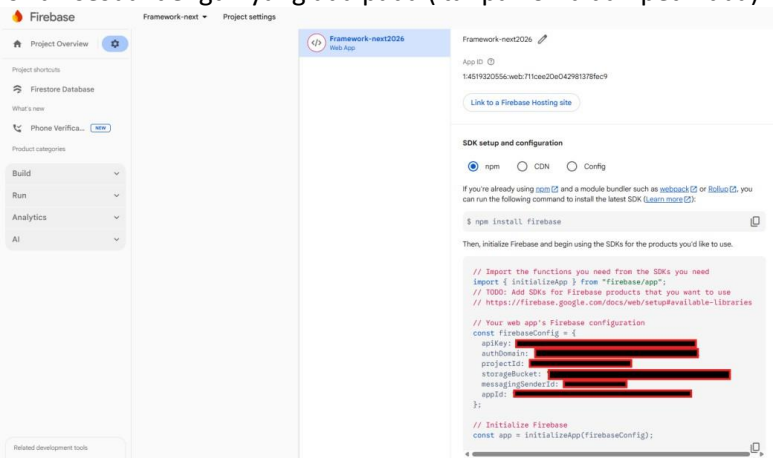
1. Buat file: .env.local



2. Modifikasi file env

   - FIREBASE_API_KEY=xxxx, FIREBASE_AUTH_DOMAIN=xxxx, FIREBASE_PROJECT_ID=xxxx, FIREBASE_STORAGE_BUCKET=xxxx, FIREBASE_MESSAGING_SENDER_ID=xxxx, FIREBASE_APP_ID=xxxx

- Isikan sesuai dengan yang ada pada (tanpa koma dan petik dua)



6.praktikum-ApiRoutes > my-app > .env.local

```
FIREBASE_API_KEY=[redacted]
FIREBASE_AUTH_DOMAIN=[redacted]
FIREBASE_PROJECT_ID=[redacted]
FIREBASE_STORAGE_BUCKET=[redacted]
FIREBASE_MESSAGING_SENDER_ID=[redacted]
FIREBASE_APP_ID=[redacted]
```

Langkah 8 – Konfigurasi Firebase

1. Modifikasi firebase.ts

6.praktikum-ApiRoutes > my-app > src > pages > utils > db > TS firebase.ts > default

```
// Import the functions you need from the SDKs you need
import { initializeApp } from "firebase/app";
// TODO: Add SDKs for Firebase products that you want to use
// https://firebase.google.com/docs/web/setup#available-libraries

// Your web app's Firebase configuration
const firebaseConfig = {
  apiKey: process.env.FIREBASE_API_KEY,
  authDomain: process.env.FIREBASE_AUTH_DOMAIN,
  projectId: process.env.FIREBASE_PROJECT_ID,
  storageBucket: process.env.FIREBASE_STORAGE_BUCKET,
  messagingSenderId: process.env.FIREBASE_MESSAGING_SENDER_ID,
  appId: process.env.FIREBASE_APP_ID
};

// Initialize Firebase
const app = initializeApp(firebaseConfig);

export default app;
```

Langkah 9 – Ambil Data dari Firestore

1. Buat file
   - ulits/db/servicefirebase.js

```
utils\db
TS firebase.ts                U
TS servicefirebase.ts         U
```

- modifikasi file servicefirebase.js

```
jobsheet7 > my-app > src > pages > utils > db > TS servicefirebase.ts > ...
import { getFirestore, collection, getDocs } from "firebase/firestore";
import app from "./firebase";

const db = getFirestore(app);

export async function retrieveProducts(collectionName: string) {
  const snapshot = await getDocs(collection(db, collectionName));
  const data = snapshot.docs.map((doc) => ({
    id: doc.id,
    ...doc.data(),
  }));
  return data;
}
```

Langkah 10 – API Mengambil Data Firebase

1. Edit pages/api/product.js:

```
6.praktikum-ApiRoutes > my-app > src > pages > api > TS produk.ts > ...
// Next.js API route support: https://nextjs.org/docs/api-routes/introduction
import type { NextApiRequest, NextApiResponse } from "next";
import { retrieveProducts } from "../utils/db/servicefirebase";

Windsurf: Refactor | Explain
type Data = {
  status: boolean;
  status_code: number;
  data: any;
};

Windsurf: Refactor | Explain | Generate JSDoc | X
export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse<Data>,
) {
  const data = await retrieveProducts("products");
  res.status(200).json({ status: true, status_code: 200, data });
}
```

2. Jalankan browser http://localhost:3000/api/produk

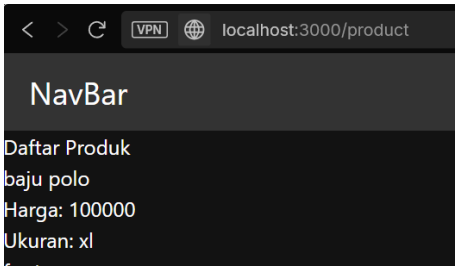
```
localhost:3000/api/produk
{
  "status": true,
  "status_code": 200,
  "data": [
    {
      "id": "fg9fDEnI2liy4IWSmLgc",
      "name": "baju polo",
      "size": "xl",
      "price": 100000
    }
  ]
}
```

3. Modifikasi index.ts pada produk sesuaikan nama typenya dan db nya

6.praktikum-ApiRoutes > my-app > src > pages > produk > index.tsx > kategori > useEffect() callback > then() callback

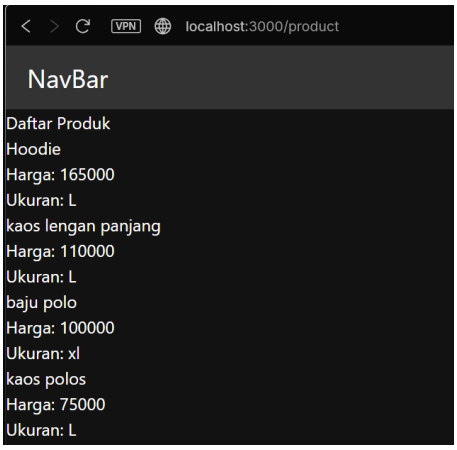
```
1  import { useRouter } from "next/router";
2  import { useEffect, useState } from "react";
3
   Windsurf: Refactor | Explain
4  type ProductType =
5  {
6      id: string;
7      name: string;
8      price: number;
9      size: string;
10   }
   Windsurf: Refactor | Explain | Generate JSDoc | X
11 const kategori = () => {
12   // const [isLogin, setIsLogin] = useState(false);
13   // const { push } = useRouter();
14   const [products, setProducts] = useState([]);
15
16   // useEffect(() => {
17   //   if (!isLogin) {
18   //     push("/auth/login");
19   //   }
20   //   },[]);
21
22   useEffect(() => {
23     fetch("/api/produk")
24       .then((response) => response.json())
25       .then((responsedata) => {
26         setProducts(responsedata.data);
27         // console.log("Data produk:", responsedata.data);
28       })
29       .catch((error) => {
30         console.error("Error fetching produk:", error);
31       });
32   }, []);
33
34
35   return (
36   <div>
37     <h1>Daftar Produk</h1>
38     {products.map((products:ProductType) => (
39     <div key={products.id}>
40       <h2>{products.name}</h2>
41       <p>Harga: {products.price}</p>
42       <p>Ukuran: {products.size}</p>
43     </div>
44   ))}
45   </div>
46   );
47 };
48
49 export default kategori;
50
```

Jalankan browser

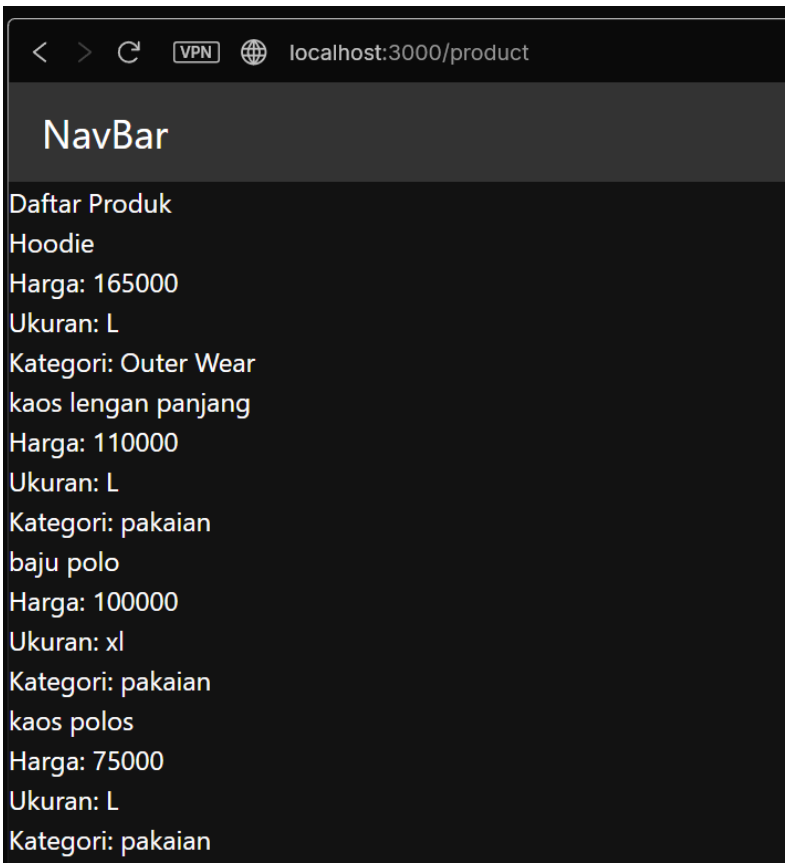


F. Tugas Praktikum

Tugas 1 (Wajib)

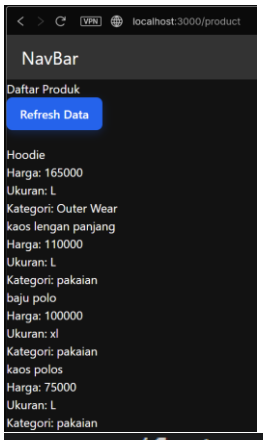
- Tambahkan minimal 3 data produk di Firestore
- Pastikan data tampil di halaman produk
- 

Tugas 2 (Wajib)

- Tambahkan field baru:
    - category
- Tampilkan category di frontend
- 

Tugas 3 (Pengayaan)

- Tambahkan tombol Refresh Data
- Gunakan fetch ulang tanpa reload halaman




```
messages/fast-refresh-reload
 GET /product 200 in 95ms (next.js: 77ms, application-code: 18ms)
 GET /product 200 in 14ms (next.js: 4ms, application-code: 9ms)
 GET /api/produk 304 in 91ms (next.js: 13ms, application-code: 78ms)
 GET /api/produk 304 in 73ms (next.js: 1322µs, application-code: 72ms)
```

G. Pertanyaan Evaluasi

1. Apa fungsi API Routes pada Next.js?
2. Mengapa .env.local tidak boleh di-push ke repository?
3. Apa perbedaan data statis dan data dinamis?
4. Mengapa Next.js disebut framework fullstack?

H. Jawaban

1. API Routes pada Next.js adalah fitur yang memungkinkan kamu membuat endpoint backend langsung di dalam folder src/pages/api, sehingga aplikasi bisa menangani request HTTP seperti GET, POST, atau DELETE tanpa perlu server terpisah.
2. File .env.local tidak boleh di-push ke repository karena biasanya berisi data sensitif seperti API key, secret, atau konfigurasi pribadi; jika masuk ke Git, informasi tersebut bisa bocor atau disalahgunakan oleh orang lain.
3. Data statis adalah konten yang sudah tetap dan tidak berubah setiap kali halaman dimuat, sedangkan data dinamis berubah berdasarkan request, pengguna, atau sumber data seperti database, sehingga konten tersebut dapat berbeda setiap kali diambil.
4. Next.js disebut framework fullstack karena selain menyediakan frontend React untuk membuat UI dan routing, ia juga menyediakan fitur backend seperti API Routes, server-side rendering, dan akses ke database, memungkinkan frontend dan backend dibangun dalam satu proyek.

•